```
# ==============================
# Practical No: 2
# IMDB Sentiment Analysis using Deep Neural Network
# ==============================

# Step 1: Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, Flatten
```

```
# Step 2: Set Parameters
vocab_size = 10000
max_length = 200
```

```
# Step 3: Load Dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000
Testing samples: 25000
```

```
# Step 4: Padding Sequences
X_train = pad_sequences(X_train, maxlen=max_length)
X_test = pad_sequences(X_test, maxlen=max_length)
```

```
# Step 5: Build Model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=32, input_length=max_length))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecat
  warnings.warn(
```

```
# Step 6: Compile Model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
# Step 7: Train Model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2
)
```

```
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 8s 39ms/step - accuracy: 0.7438 - loss: 0.4896 - val_accuracy: 0.8560 - val_loss: 0.3242
Epoch 2/5
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 6s 36ms/step - accuracy: 0.9393 - loss: 0.1701 - val_accuracy: 0.8604 - val_loss: 0.3290
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 6s 41ms/step - accuracy: 0.9911 - loss: 0.0408 - val_accuracy: 0.8514 - val_loss: 0.4095
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 6s 36ms/step - accuracy: 0.9989 - loss: 0.0088 - val_accuracy: 0.8636 - val_loss: 0.4453
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 7s 42ms/step - accuracy: 0.9998 - loss: 0.0024 - val_accuracy: 0.8592 - val_loss: 0.4807
```

```
# Step 8: Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)
print("\nTest Accuracy:", accuracy)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8300 - loss: 0.6068

Test Accuracy: 0.8299599885940552
```